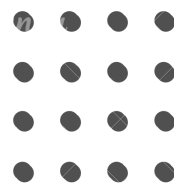




VMES Grinds

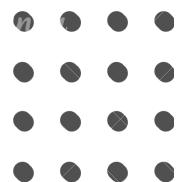




VMES Grinds

Pilot Program - Week 6 - Day 1

Graph • BFS • DFS





LeetCode: 1971

Find if Path Exists in Graph



Moe Myint Mo San



1971. Find if Path Exists in Graph

Solved

Easy

Topics

Companies

There is a **bi-directional** graph with n vertices, where each vertex is labeled from 0 to $n - 1$ (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex u_i and vertex v_i . Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

Given `edges` and the integers n , `source`, and `destination`, return `true` if there is a **valid path** from `source` to `destination`, or `false` otherwise.



Constraints:

- $1 \leq n \leq 2 * 10^5$
- $0 \leq \text{edges.length} \leq 2 * 10^5$
- $\text{edges}[i].\text{length} == 2$
- $0 \leq u_i, v_i \leq n - 1$
- $u_i \neq v_i$
- $0 \leq \text{source}, \text{destination} \leq n - 1$
- There are no duplicate edges.
- There are no self edges.



Problem Explanation Thought Process





Initial Thought Process

- This is a reachability problem.
- Graph is:
 - Undirected
 - Unweighted
- We only care about:
 - Whether destination can be reached.
- Use graph traversal starting from source.



Algorithm Evolution Overview



Approach	Time	Space	Notes
BFS / DFS	$O(n + m)$	$O(n + m)$	Standard reachability



Breadth-First Search (BFS)





Idea:

- Convert the edge list into an adjacency list.
- Start BFS from source.
- Visit all reachable nodes.
- If destination is found → return True.

Why It Works

- BFS explores all nodes reachable from source.
- If destination is reachable, BFS will eventually visit it.
- visited prevents:
 - Infinite loops
 - Re-processing nodes



```
from collections import defaultdict, deque

class Solution(object):
    def validPath(self, n, edges, source, destination):
        """
        :type n: int
        :type edges: List[List[int]]
        :type source: int
        :type destination: int
        :rtype: bool
        """
        if source == destination:
            return True

        graph = defaultdict(list)
        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        queue = deque([source])
        visited = set([source])

        while queue:
            node = queue.popleft()
            for neighbor in graph[node]:
                if neighbor == destination:
                    return True

                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

        return False
```



Time Complexity (Brute Force)

- Time: $O(n + m)$
 - Each node visited once
 - Each edge processed once
- Space: $O(n + m)$
 - Adjacency list
 - Queue + visited set



Complexity Comparison





Approach	Time	Space	Notes
BFS / DFS	$O(n + m)$	$O(n + m)$	Optimal reachability



Key Takeaway





This is a pure graph reachability problem:

- Undirected + unweighted graph
- BFS (or DFS) is the correct pattern
- Always:
 - Build adjacency list
 - Track visited nodes
 - Exit early when destination is found



Thank You !

